



兰州工业学院

智能仪器仪表大作业

(2024 —2025 学年第 1 学期)

学 院 电子信息工程学院

专业班级 专升本电信 23

学 号 ZB2302010116

姓 名 姜 勇

完成日期 2024.12.1~2024.12.20

目录

1. 引言.....	1
1.1 背景与动机.....	1
1.2 目标与意义.....	1
2. 温度检测与控制的基本方法.....	1
2.1 温度检测方法.....	1
2.2 温度控制方法.....	2
3. 总体方案设计.....	2
3.1 系统框图.....	2
3.2 系统工作原理.....	2
3.2.1 温度采集与处理.....	2
3.2.2 温度上下限设置.....	3
3.2.3 温控调节功能.....	3
3.2.4 温度显示.....	3
3.2.5 报警功能.....	3
3.2.6 串口通信.....	4
3.2.7 整体工作流程.....	4
4. 各功能电路设计.....	4
4.1 温度采集电路设计.....	4
4.2 按键输入电路设计.....	5
4.3 OLED 显示电路设计.....	5
4.4 串口通信电路设计.....	6
4.5 继电器控制电路设计.....	6
4.6 蜂鸣器报警电路设计.....	7
4.7 系统电路图.....	7
5. 软件设计.....	8
5.1 总体软件设计.....	8
5.2 温度采集模块设计.....	8
5.3 按键输入模块设计.....	8
5.4 OLED 显示模块设计.....	9
5.5 串口通信模块设计.....	9
5.6 继电器控制模块设计.....	9
5.7 蜂鸣器报警设计.....	10
5.8 系统主流程设计.....	10
5.9 模块流程图.....	11
6. 系统仿真与运行效果.....	14
6.1 Proteus 仿真.....	14
6.2 仿真结果分析.....	18
7. 结论.....	18
附件 A 源程序.....	20

设计一个以单片机为控制核心的单回路温度控制系统，主要包括温度检测、温度控制、参数显示、报警指示、通信等单元电路。温度控制器与上位机配合可构成一套完整的温度监控系统。主要功能包括：设定温度、实时温度显示；温度上、下限报警；温度上、下限报警值设定；目标温度值设定。

1. 分析温度检测、控制的基本方法；

2. 完成单回路温度控制器的总体方案设计，包括工作原理、系统的基本组成等；

3. 确定单回路温度控制器的操作模式。

3. 各功能电路设计。包括：温度检测电路、温度控制电路和控制器和 PC 之间数据通信电路等；

4. 程序设计。包括：主控制程序、显示按键处理程序、与上位机数据通信程序等。

5. 撰写设计报告。

具体步骤如下：

1. 完成对设计项目的需求分析和系统设计分析；

2. 设计电路并绘制原理图，理清每一部分电路的原理及功能；

3. 在 Keil 软件完成源程序编写、仿真及调试；

4. 依据设计的原理图使用 Proteus 等软件实现仿真；

5. 撰写设计报告。

1. 引言

1.1 背景与动机

随着物联网技术的快速发展，基于单片机的温度监测与控制系统在多个领域得到了广泛应用。温度监控系统可以实时监测环境温度并进行控制，广泛应用于工业自动化、智能家居和环境监测等领域。对于温控系统的自动化和高效性，采用单片机实现温度数据采集、控制和显示具有重要意义。

1.2 目标与意义

本文设计的温度监测与控制系统通过 STM32F103C8T6 单片机进行温度数据采集与处理，采用 DS18B20 传感器进行温度采集，利用 4 针脚 OLED 屏幕显示实时温度，独立式按键进行温度上下限的设置。当温度超过上下限时，通过继电器控制加热或散热设备，并通过蜂鸣器报警。此外，系统通过串口通信接收和发送温度数据及上下限设置。该系统具有较高的应用价值，能够广泛应用于温控设备的温度监测和管理中。

2. 温度检测与控制的基本方法

2.1 温度检测方法

温度检测的常见方法有：热电偶法、RTD 法、热敏电阻法、数字温度传感器

法等。本项目中采用的是 DS18B20 数字温度传感器，其优点是无需校准，使用方便且具有较高的精度。DS18B20 通过 1-Wire 协议与单片机进行通信，能够输出数字信号，避免了模拟信号的噪声干扰。

2.2 温度控制方法

温度控制系统的基本方法通常包括开关控制和 PID 控制两种。本项目采用的是开关控制方法，即通过设定温度上下限来启动继电器控制加热或散热设备。当温度超过设定的上限或下限时，继电器会启动相应的加热或散热装置来调节温度。

3. 总体方案设计

3.1 系统框图

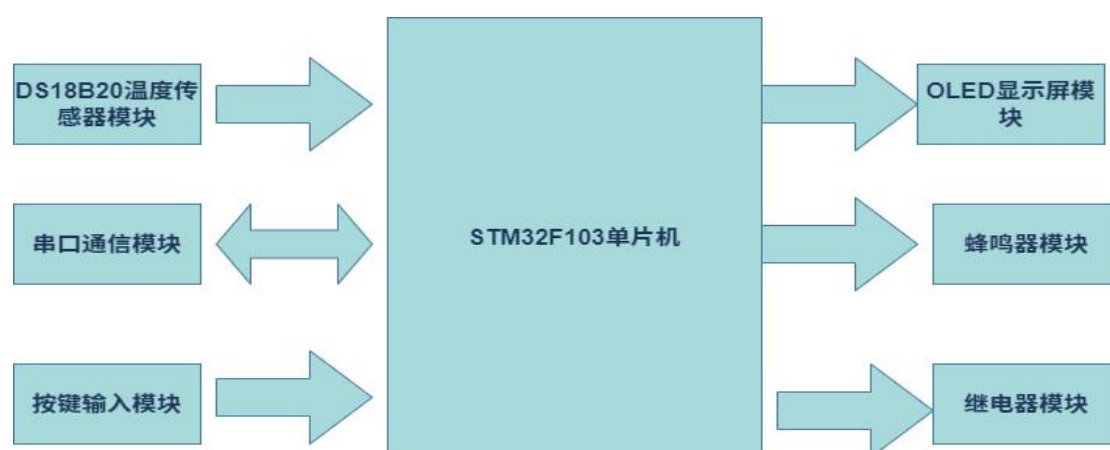


图 3-1 系统框图

3.2 系统工作原理

该温度监测与控制系统的工作原理主要包括温度数据采集、数据处理与控制、温度显示、报警及温控系统的自动调节功能。整个系统的工作流程如下：

3.2.1 温度采集与处理

系统通过 DS18B20 数字温度传感器实时采集环境温度。DS18B20 传感器使用 1-Wire 协议与 STM32F103C8T6 单片机通信。温度传感器会将环境温度转换为数字信号，并通过单片机进行处理。每次温度采集后，传感器将测得的温度数据传送至 STM32 单片机的输入端口。由于 DS18B20 提供数字输出，无需模拟信号的转换，因此可提高测量精度，减少噪声干扰。

单片机通过 I/O 端口 或 USART 串口 接收从传感器返回的温度值，并进行

处理，温度数据会被转换为可显示的单位（如 $^{\circ}\text{C}$ ）。经过计算后的温度值将被传输到显示模块（OLED 显示屏），用于实时显示当前环境温度。

3.2.2 温度上下限设置

温度上下限通过按键进行输入设置。用户可以通过按键设定温度上限和温度下限，确保系统在预定温度范围内工作。当用户按下某个按键时，单片机会接收按键信号并解析，自动更新温度的上下限。

设置完成后，单片机会根据实时温度与上下限进行对比判断，如果当前温度超过设定的上下限，系统将启动温度控制功能。此功能包括通过继电器控制加热或散热设备来调节温度，确保环境温度处于设定范围内。

3.2.3 温控调节功能

系统的核心控制部分是温度调节模块。当实时温度超过设定的上限或下限时，单片机会通过继电器控制外部加热或散热设备的开关。

- 加热功能：当温度低于设定下限时，继电器控制加热设备（如电热器）启动，加热环境直到温度回升至上限范围。
- 散热功能：当温度超过设定上限时，继电器控制散热设备（如风扇或冷却系统）启动，帮助环境温度降低至设定下限范围内。

在调节过程中，系统通过继电器开关实现对加热或散热设备的控制，继电器的启闭直接决定了加热或散热装置的工作状态。

3.2.4 温度显示

实时温度信息通过 OLED 显示屏 显示。OLED 显示屏与 STM32F103C8T6 单片机通过 I2C 接口连接，能够清晰显示当前的温度值以及温度上下限。如果温度超过或低于设定范围，OLED 显示屏会显示警告信息，并提示用户温度状态。

3.2.5 报警功能

为了确保系统的安全运行，当温度超出设定的范围时，系统会通过蜂鸣器发出警报，提醒用户温度已超限。蜂鸣器的启停由单片机控制，在温度超限时，蜂鸣器会发出声音，直到系统恢复正常。

3.2.6 串口通信

除了本地控制和显示功能外，系统还具有串口通信功能，通过 USART 串口接口与外部设备（如 PC 或上位机）进行数据交换。通过串口，系统可以实时向外部设备发送当前温度值，并接收上下限设置。当用户通过串口向单片机发送 "T:xx.xx" 或 "L:xx.xx" 格式的数据时，系统会解析并更新当前的温度上下限设置。

3.2.7 整体工作流程

整个系统的工作流程为：

1. 温度采集：DS18B20 传感器通过 1-Wire 协议采集当前温度。
2. 数据处理：单片机接收传感器数据并处理，转换为可显示的格式。
3. 温度显示：OLED 显示屏实时显示当前温度及上下限设置。
4. 上下限判断：单片机根据当前温度与用户设置的上下限进行比较，判断是否需要控制加热或散热。
5. 温控调节：当温度超过上下限时，通过继电器控制加热或散热设备工作。
6. 报警功能：如果温度超限，蜂鸣器发出警报提醒用户。
7. 串口通信：通过串口与外部设备交换数据，实现远程温度监控和上下限设置。

4. 各功能电路设计

4.1 温度采集电路设计

DS18B20 温度传感器通过 1-Wire 协议与 STM32F103C8T6 单片机进行通信。其电路设计如下：

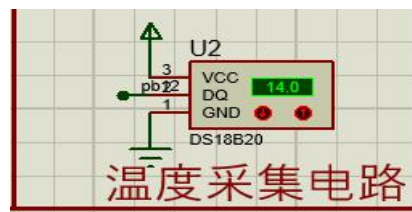


图 4-1 温度采集电路

原理：DS18B20 提供数字信号输出，避免了模拟信号的噪声干扰。其数据引脚通过一个上拉电阻连接到 STM32 的 GPIO 端口，用于通信。

优点：无需校准，精度高，使用方便。

4.2 按键输入电路设计

本系统采用独立式按键实现用户输入，电路如下：

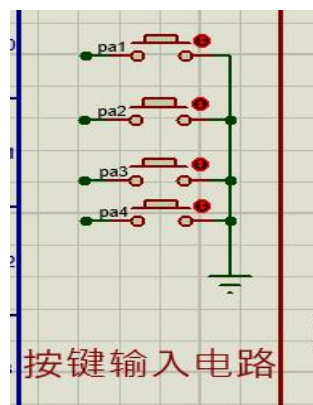


图 4-2 按键输入电路

原理：每个按键与 STM32 的一个 GPIO 端口连接，当按键按下时，通过电平变化检测输入。

功能：支持设置温度上下限，简单可靠。

4.3 OLED 显示电路设计

OLED 显示屏与 STM32 通过 I2C 接口通信，电路如下：

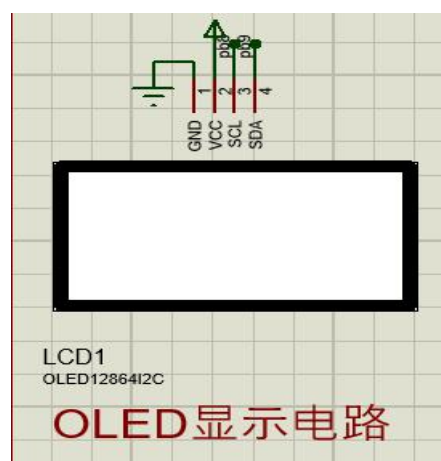


图 4-3 显示电路

原理：OLED 显示屏通过两根信号线（SCL、SDA）与 STM32 连接，用于传输

显示数据。

功能：显示实时温度、上下限设定值以及报警状态。

4.4 串口通信电路设计

系统采用 USART 接口实现与外部设备（如 PC）的数据交换，电路如下：

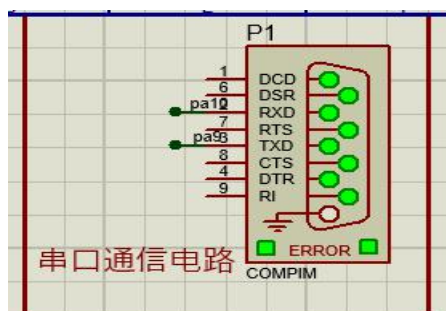


图 4-4 串口通信电路

原理：STM32 通过 USART 发送温度数据并接收上下限设定指令。

功能：支持远程监控和设置。

4.5 继电器控制电路设计

继电器用于控制加热或散热设备，电路如下：

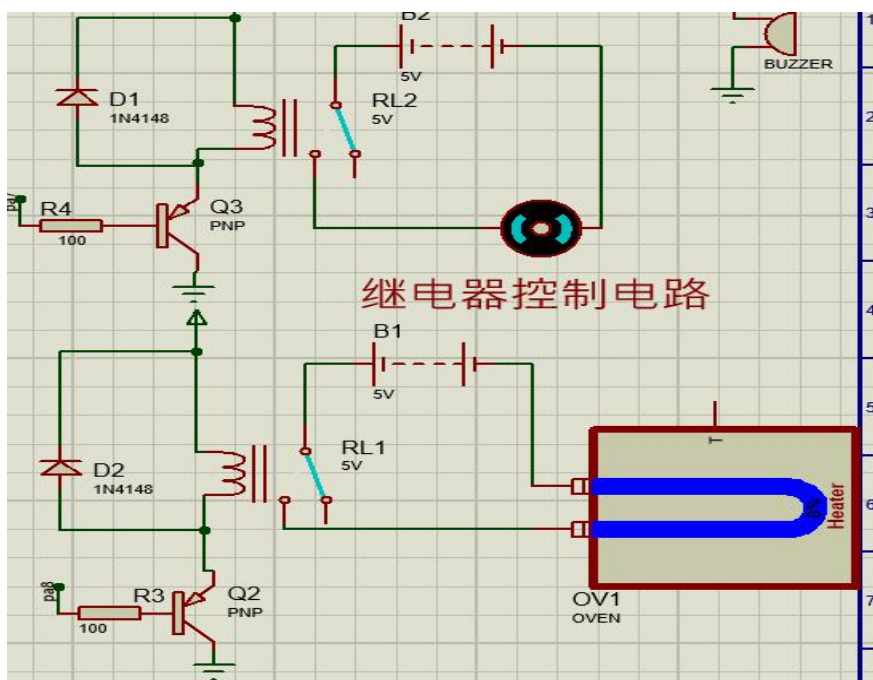


图 4-5 继电器控制电路

原理：STM32 通过 GPIO 端口控制继电器的开闭，以启动或停止外部设备。

功能：根据温度上下限调节环境温度。

4.6 蜂鸣器报警电路设计

蜂鸣器用于温度超限报警，电路如下：

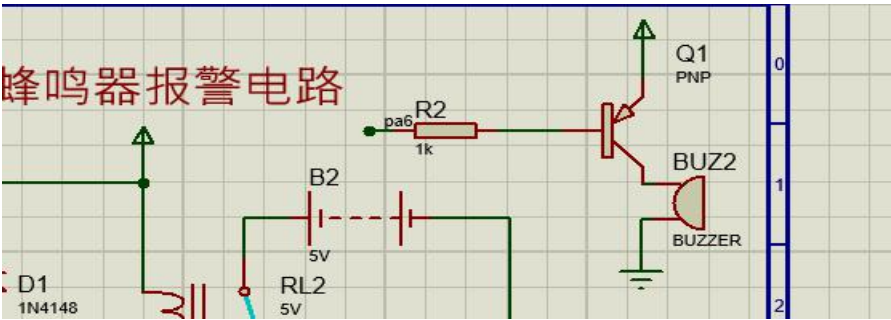


图 4-6 蜂鸣器报警电路

原理：当温度超限时，STM32 输出高电平触发蜂鸣器发声。

功能：提醒用户温度异常。

4.7 系统电路图

整体电路设计集成了上述所有功能模块，包括传感器、按键、显示屏、串口、继电器及蜂鸣器。

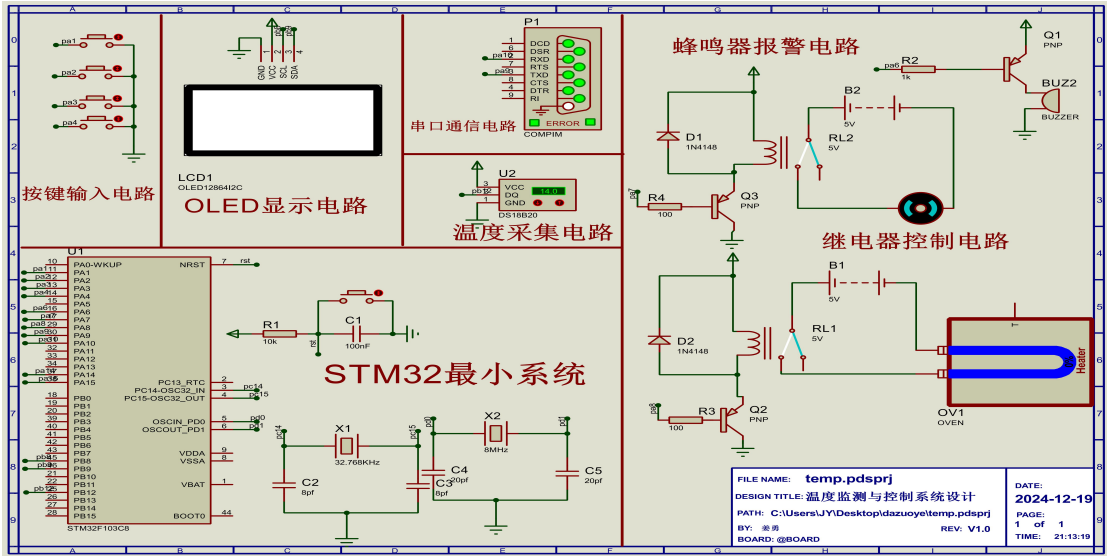


图 4-7 系统总体电路图

5. 软件设计

5.1 总体软件设计

软件设计以模块化为核心思想，各功能模块分工明确，主程序通过调用各模块函数实现系统的整体协调工作。主要包括温度采集模块、按键输入模块、显示模块、串口通信模块、继电器控制模块和蜂鸣器报警模块。

系统的软件结构分为以下几个层次：

驱动层：与硬件设备直接交互，为上层提供接口。

逻辑层：实现具体功能逻辑，如温度数据处理、阈值判断等。

接口层：处理外部通信和用户交互。

5.2 温度采集模块设计

功能：通过 DS18B20 温度传感器实时采集温度数据。

主要步骤：

1. 初始化 1-Wire 总线，配置 DS18B20 工作模式。
2. 发送温度转换命令，等待传感器完成温度测量。
3. 读取传感器寄存器中的温度数据，解析并转换为摄氏度格式。
4. 返回温度数据供其他模块使用。

注意事项：定时读取温度数据，确保数据实时性，同时处理可能的通信异常。

5.3 按键输入模块设计

功能：捕获用户输入的温度上下限设定值。

主要步骤：

1. 定时扫描按键状态，通过 GPIO 端口读取按键电平信号。
2. 识别按键按下事件，进行去抖处理，防止误触发。
3. 根据按键编号和当前状态更新温度上下限。
4. 实时反馈设定结果至显示模块。

扩展功能：支持长按功能，例如快速调整温度阈值。

5.4 OLED 显示模块设计

功能：显示实时温度、设定值及报警状态。

主要步骤：

1. 初始化 OLED 显示模块，设置 I2C 通信参数。
2. 格式化温度和状态数据为字符串。
3. 调用驱动函数更新显示内容，包括当前温度、上下限设定值和报警状态。
4. 在温度超限时，高亮显示报警信息。

扩展功能：提供简单图形界面，提升显示效果。

5.5 串口通信模块设计

功能：实现与上位机的数据交互。

主要步骤：

1. 初始化 USART 接口，配置波特率、数据位、校验位等参数。
2. 监听串口缓冲区，接收来自上位机的指令数据（例如："T:30.0"设定上限温度）。
3. 解析指令，提取目标数据并更新系统设定。
4. 定时发送当前温度及状态数据至上位机。

注意事项：处理通信错误，确保数据完整性。

5.6 继电器控制模块设计

功能：根据实时温度与设定上下限控制加热或散热设备。

主要步骤：

1. 获取当前温度数据，与上下限比较。
2. 如果温度低于下限，输出高电平至继电器，启动加热设备；如果高于上限，启动散热设备。
3. 在温度回归正常范围后关闭继电器，停止设备工作。

4. 实现状态切换延时，避免频繁启停对设备造成损害。

5.7 蜂鸣器报警设计

功能：在温度超限时发出声响警报。

主要步骤：

1. 检测当前温度是否超过上下限设定值。
2. 如果超限，控制 GPIO 输出高电平触发蜂鸣器响起。
3. 温度恢复正常后停止蜂鸣器报警。
4. 提供手动解除报警功能，增强用户操作灵活性。

5.8 系统主流程设计

系统的主程序以循环任务形式运行，各模块协调工作，具体流程如下：

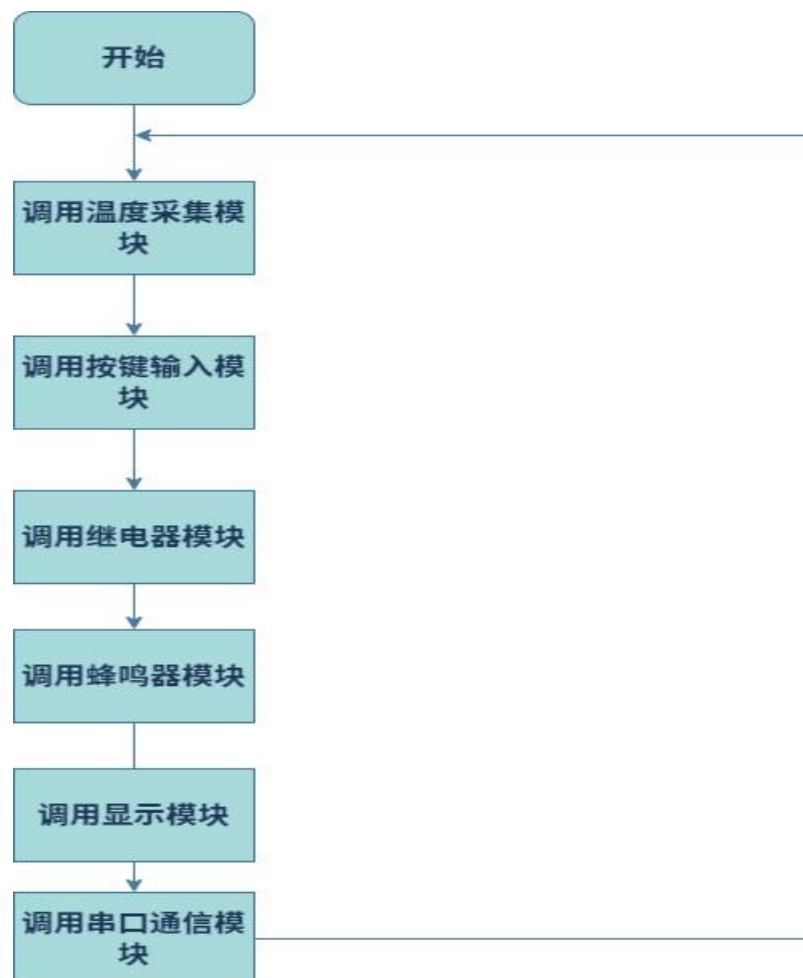


图 5-1 系统主流程图

5.9 模块流程图

温度采集显示模块：

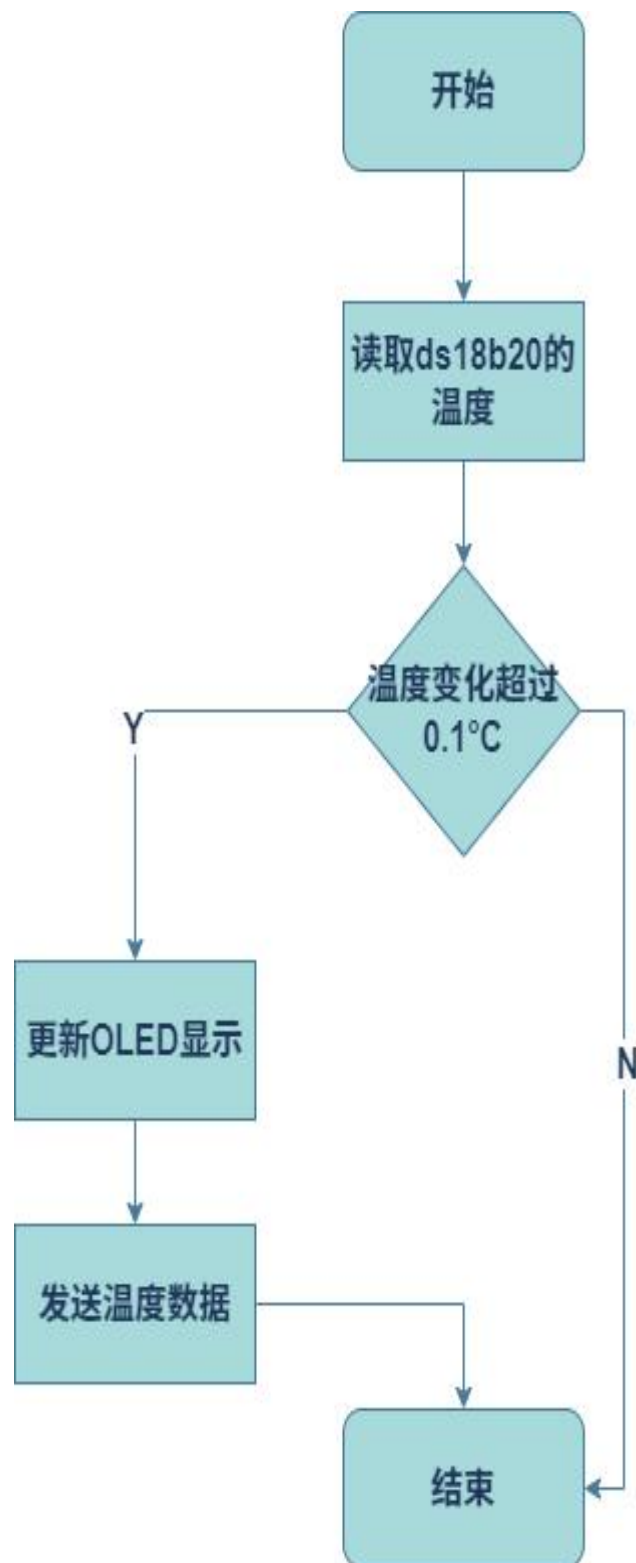


图 5-2 温度采集显示流程图

按键输入模块：

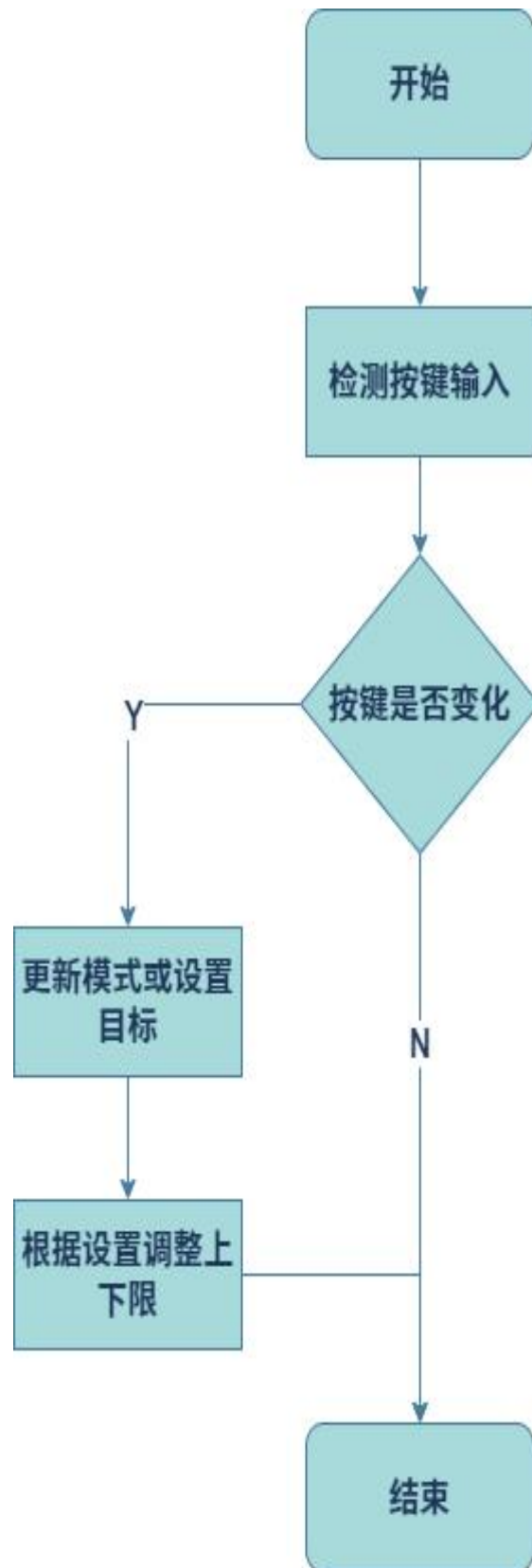


图 5-3 按键输入流程图

串口通信模块：

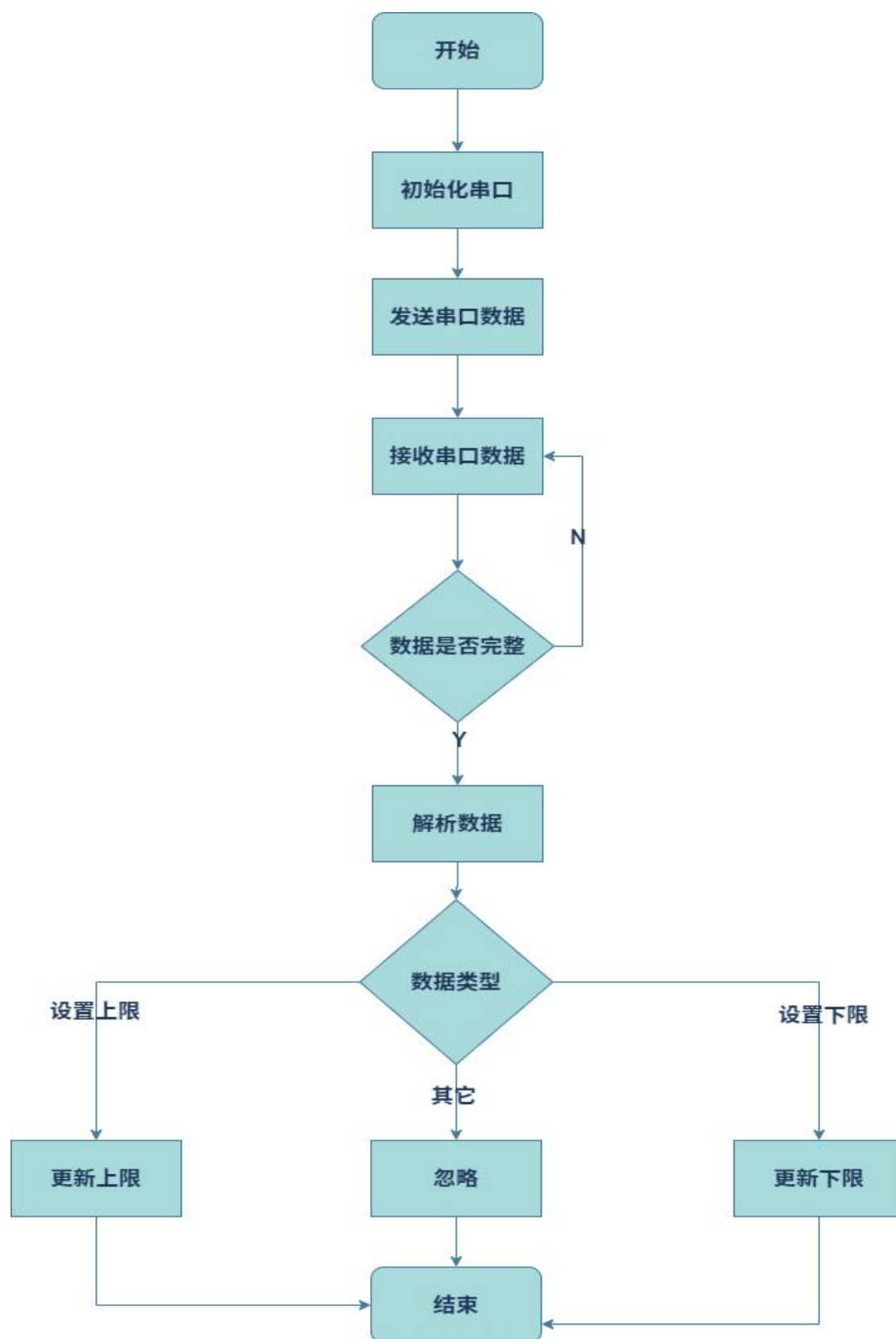


图 5-4 串口通信流程图

报警控制模块：

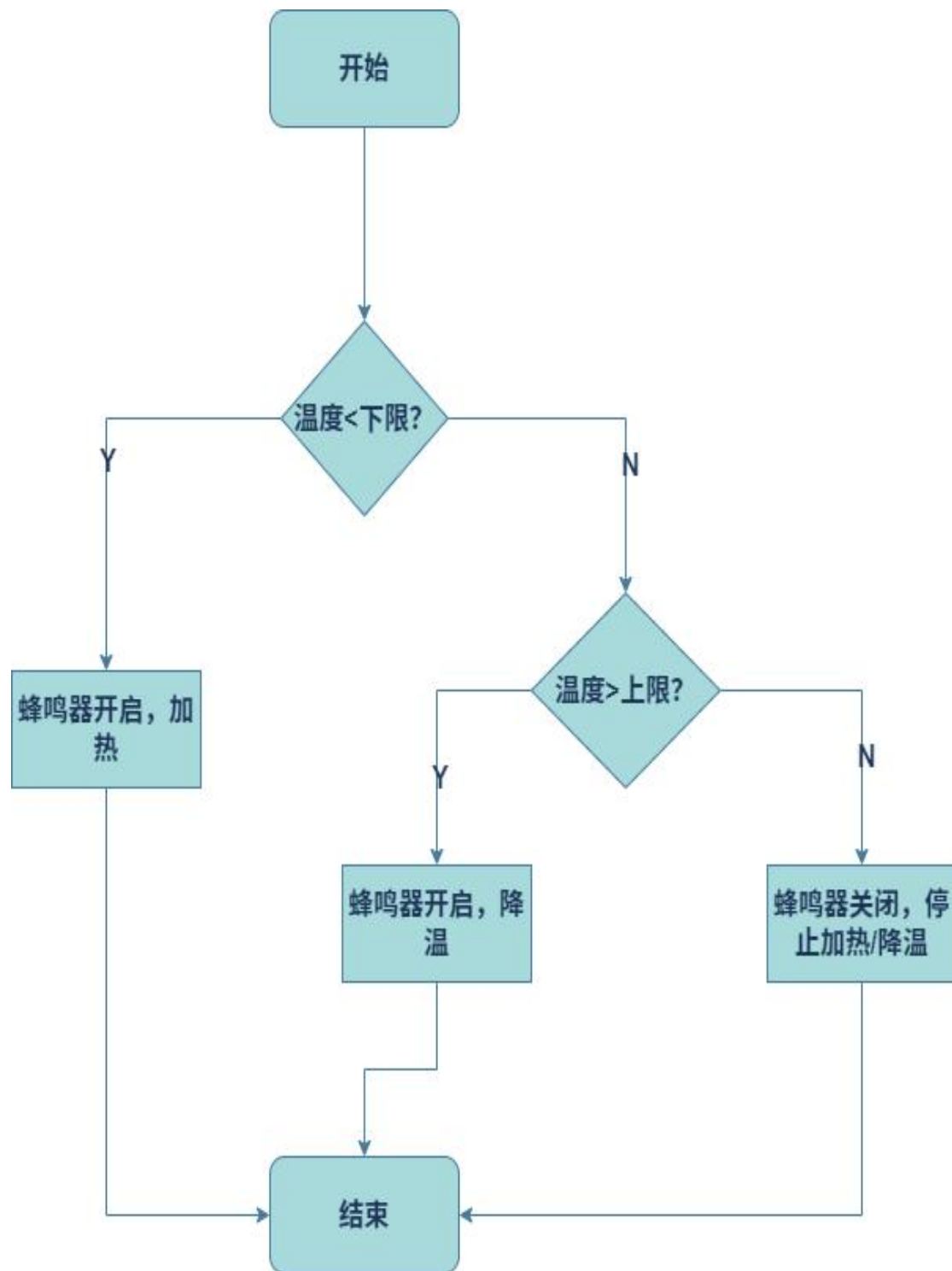


图 5-5 报警控制流程图

6. 系统仿真与运行效果

6.1 Proteus 仿真

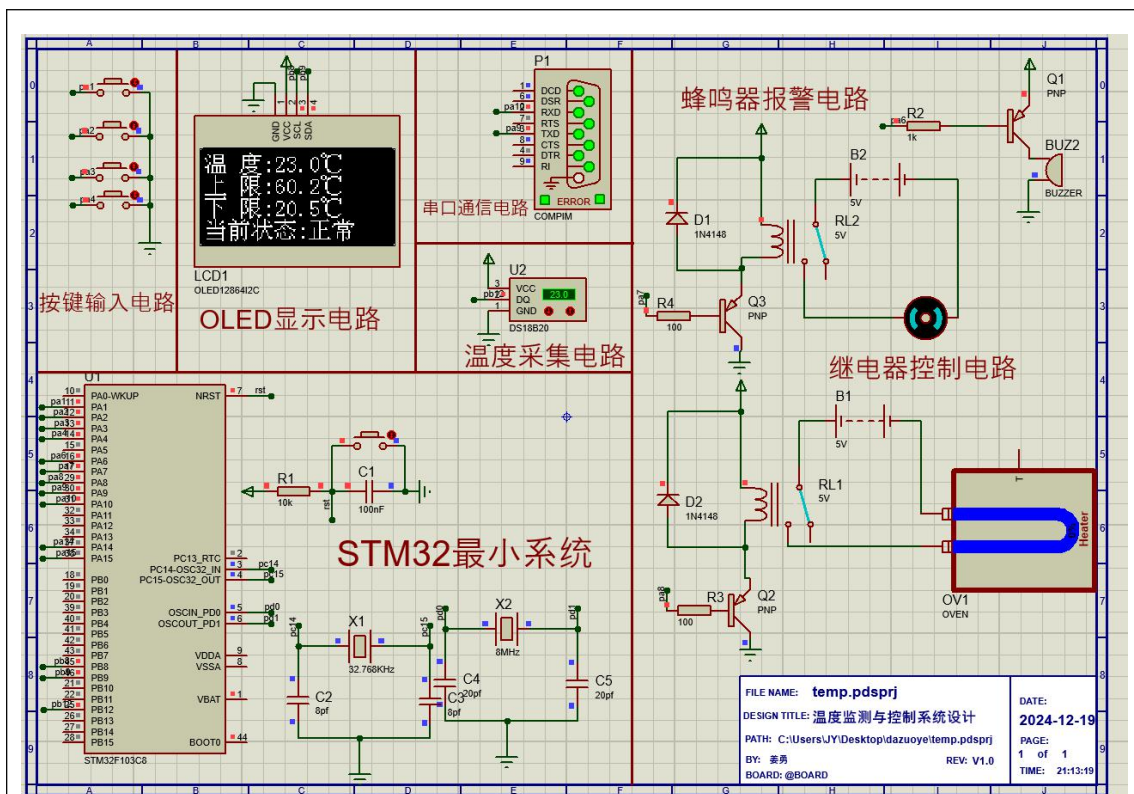


图 6-1 系统正常运行结果

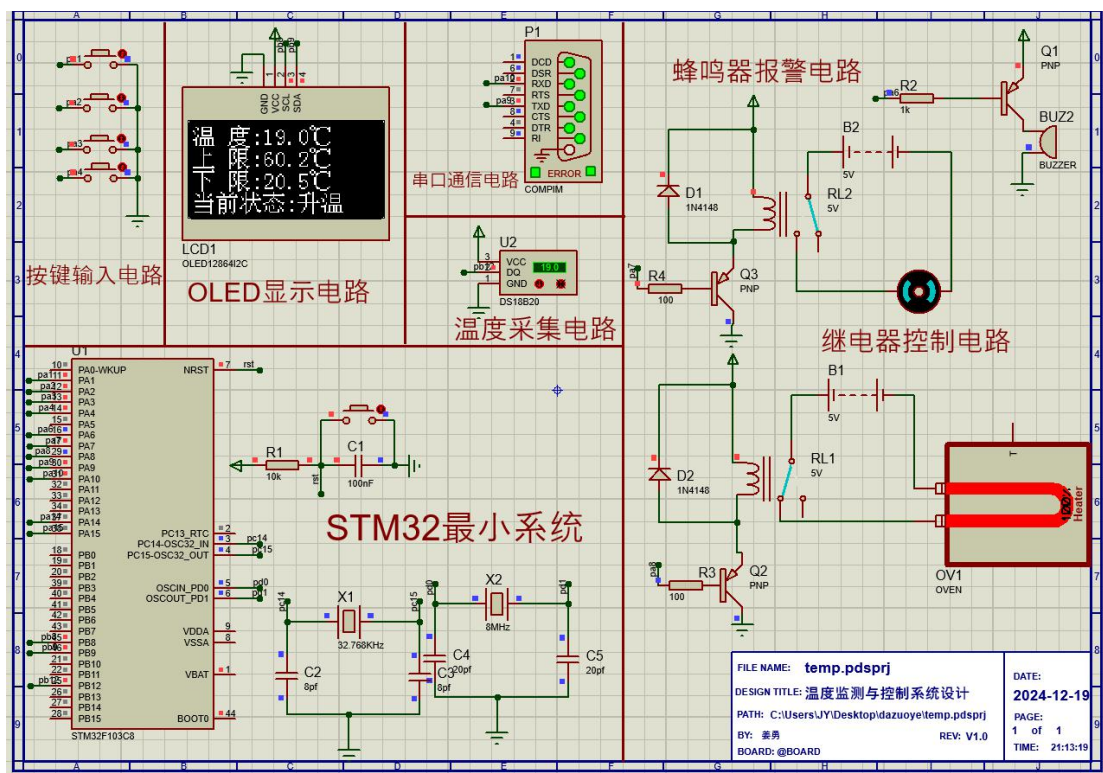


图 6-2 温度过低升温中

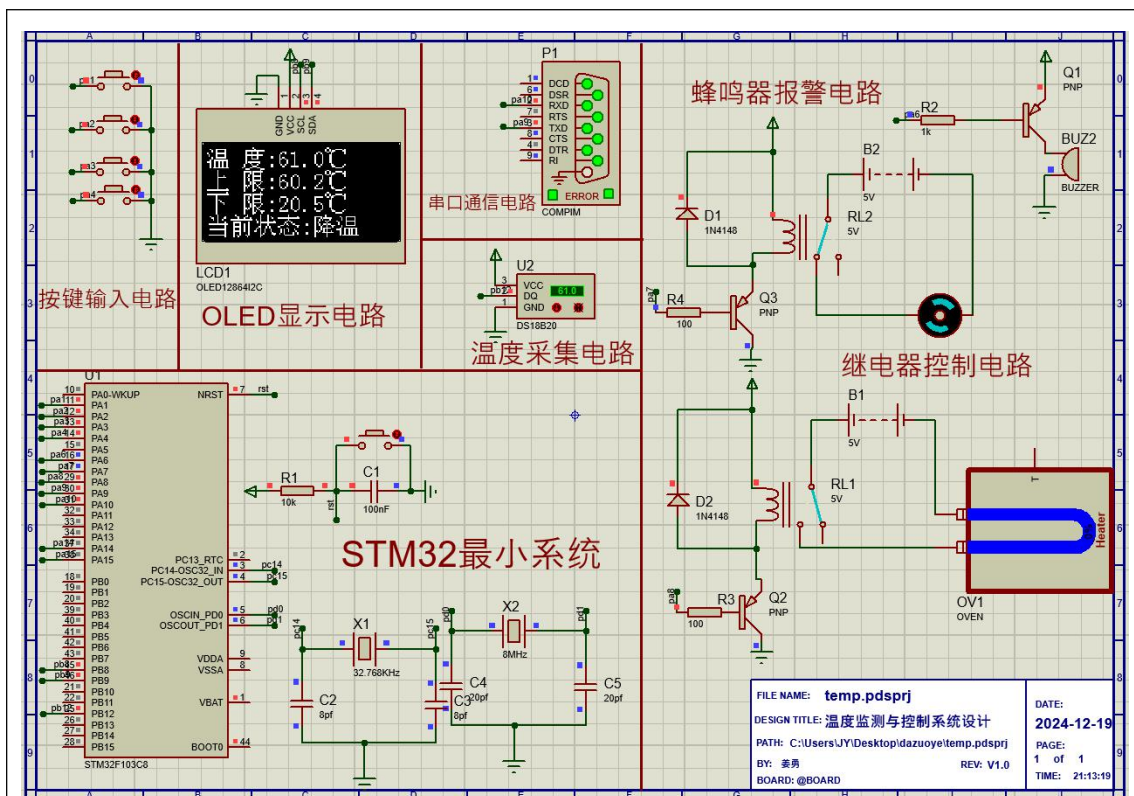


图 6-3 温度过高降温中

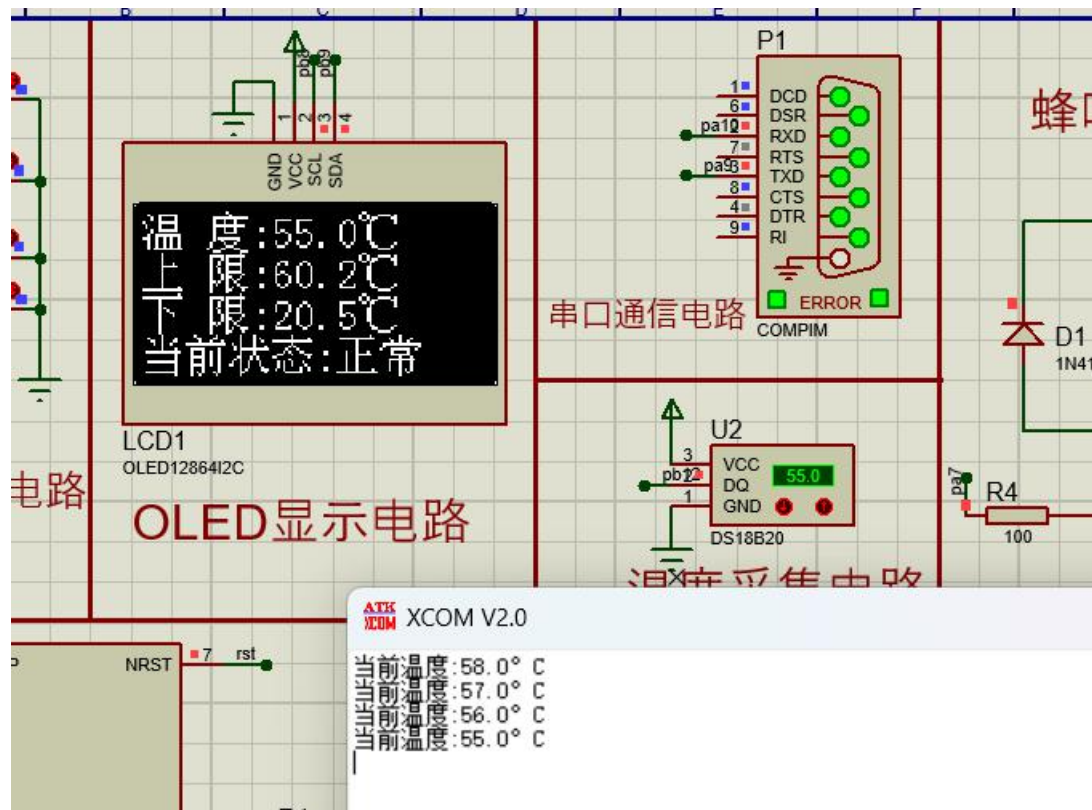


图 6-4 发送温度在上位机显示

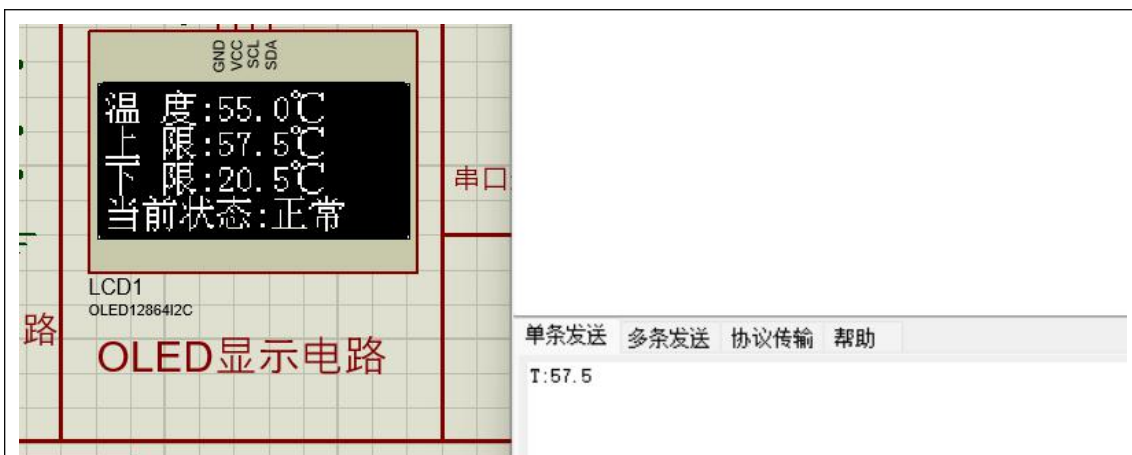


图 6-5 上位机发送设置上限

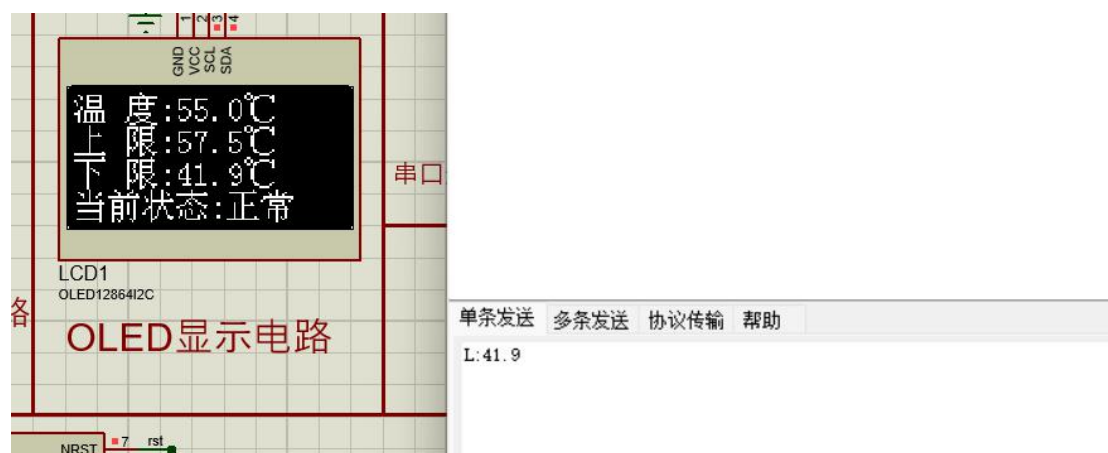


图 6-6 上位机发送设置下限

通过 Proteus 仿真，验证了系统的功能完整性和硬件设计的可靠性。仿真环境中集成了 STM32F103C8T6、DS18B20、OLED 显示屏、继电器、蜂鸣器以及按键模块，以模拟实际运行场景。以下是仿真中的主要功能表现：

实时温度采集：DS18B20 传感器稳定输出数字信号，STM32 能够正确接收并处理数据。

显示模块工作：OLED 屏幕实时更新当前温度及上下限设定值，超限时显示报警状态。

继电器动作：温度超限时，继电器切换状态，模拟加热或散热设备的启动和关闭。

报警反馈：蜂鸣器在温度异常时发出声响，提示用户及时处理。

用户交互：按键输入上下限值的设置功能正常，串口通信模块也能正确处理

指令和数据传输。

6.2 仿真结果分析

1. 功能实现效果：

系统能够在不同温度范围内稳定运行。传感器、显示屏和控制模块之间的通信流畅。

超限报警功能有效，蜂鸣器及时提醒用户。

2. 交互性能验证：

串口通信清晰且无丢包现象，确保上下限设定值准确传输。

按键输入响应迅速，设置数据正确传递到主控制器。

3. 整体性能表现：

电路设计合理，硬件运行无明显发热或故障。

软件逻辑清晰，各模块协同工作良好。

4. 改进建议：

优化继电器切换时间，避免频繁操作。

增强异常情况处理能力，例如添加更详细的错误提示。

通过 Proteus 仿真，系统的设计目标得到全面验证，展示了硬件电路和软件设计的高度一致性，为后续的硬件实物制作提供了重要的技术支持。

7. 结论

本文详细设计并实现了一种基于 STM32F103C8T6 单片机的温度监测与控制系统，该系统具备温度检测、控制、参数显示、报警指示以及通信等功能。通过采用 DS18B20 数字温度传感器进行温度采集，OLED 显示屏进行数据展示，继电器用于控制加热或散热设备，并利用蜂鸣器进行超温报警。此外，本系统还实现了串口通信功能，使得用户可以通过上位机远程监控和设置温度上下限。

从设计之初到最终完成，整个项目经历了需求分析、总体方案设计、电路设计、软件编程、仿真测试等多个阶段。在硬件方面，精心挑选了适合本项目的元

器件，如高精度的数字温度传感器 DS18B20、低功耗且显示效果良好的 OLED 屏幕等，确保了系统的稳定性和可靠性。同时，在软件设计中，采用了模块化的设计思想，将系统划分为多个独立的功能模块，如温度采集模块、按键输入模块、显示模块、串口通信模块、继电器控制模块以及蜂鸣器报警模块等，这不仅简化了程序结构，也提高了代码的可读性和维护性。

在实际运行过程中，系统表现出了良好的性能。温度采集准确无误，能够实时反映环境温度变化；温度上下限设定灵活方便，满足不同应用场景下的需求；当温度超出预设范围时，系统能迅速作出反应，启动相应的加热或散热装置进行调节，并通过蜂鸣器及时发出警报提醒用户。此外，通过串口通信功能，还可以轻松地与外部设备交换数据，实现了远程监控的目的。

总之，本项目成功地构建了一个功能齐全、性能优越的温度监测与控制系统。它不仅适用于工业自动化领域中的过程控制，也可以广泛应用于智能家居、农业温室等领域。未来的工作可以着眼于进一步优化现有算法，探索更多智能化的控制方法，比如自适应控制、模糊逻辑控制等，以期获得更好的用户体验和更高的经济效益。此外，随着物联网技术的发展，考虑将本系统接入互联网平台，实现更大范围内的互联互通，也是值得研究的方向之一。

综上所述，本次设计不仅达到了预期目标，而且为后续的研究提供了宝贵的经验和技术支持。希望在未来的工作中能够继续深化对这类智能仪器仪表的研究，不断推动其向更加智能化、网络化的方向发展。

附件 A 源程序

```
/**
 * @file ds18b20.c
 * @brief DS18B20 温度传感器控制模块的实现文件
 *
 * 该文件包含了 DS18B20 温度传感器初始化、读取温度值以及显示当前温度的
 * 功能实现。
 */
#include "ds18b20.h"
#include "Delay.h"
#include "OLED.h" // 假设 OLED 显示函数在这个头文件中定义
#include "port.h" // 假设串口发送函数在这个头文件中定义

float temperature = 20;

/**
 * @brief 配置 DS18B20 使用的 GPIO 引脚
 */
static void DS18B20_GPIO_Config(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    // 使能 GPIO 时钟

    RCC_APB2PeriphClockCmd(DS18B20_CLK, ENABLE);

    // 配置 DS18B20 数据引脚为推挽输出模式

    GPIO_InitStructure.GPIO_Pin = DS18B20_PIN;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(DS18B20_PORT, &GPIO_InitStructure);
}
```

```

}

/**
 * @brief 初始化 DS18B20 使用的 GPIO 引脚
 */
void DS18B20_Init_GPIO(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    // 使能 GPIO 时钟

    RCC_APB2PeriphClockCmd(DS18B20_CLK, ENABLE);

    // 配置 DS18B20 数据引脚为推挽输出模式

    GPIO_InitStructure.GPIO_Pin = DS18B20_PIN;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(DS18B20_PORT, &GPIO_InitStructure);
}

/**
 * @brief 将 DS18B20 数据引脚配置为上拉输入模式
 */
static void DS18B20_Mode_IPU(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    // 配置 DS18B20 数据引脚为上拉输入模式

    GPIO_InitStructure.GPIO_Pin = DS18B20_PIN;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_Init(DS18B20_PORT, &GPIO_InitStructure);
}

```

```

/**
 * @brief 将 DS18B20 数据引脚配置为推挽输出模式
 */
static void DS18B20_Mode_Out_PP(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    // 配置 DS18B20 数据引脚为推挽输出模式

    GPIO_InitStructure.GPIO_Pin   = DS18B20_PIN;

    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_Out_PP;

    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(DS18B20_PORT, &GPIO_InitStructure);
}

/**
 * @brief 向 DS18B20 发送复位脉冲
 */
static void DS18B20_Rst(void)
{
    DS18B20_Mode_Out_PP(); // 设置为推挽输出模式

    // 拉低数据线 480us

    DS18B20_DATA_OUT(LOW);

    Delay_us(750);

    DS18B20_DATA_OUT(HIGH); // 拉高数据线

    Delay_us(15);           // 等待 15~60us
}

/**
 * @brief 检测 DS18B20 的存在性

```

```

* @return 0 表示存在, 1 表示不存在
*/
static u8 DS18B20_Presence(void)
{
    u8 pulse_time = 0;
    DS18B20_Mode_IPU(); // 设置为上拉输入模式
    /* 等待 DS18B20 拉低数据线
    * 存在时 60~240us 拉低
    * 存在时 15~60us 拉高
    */
    while (DS18B20_DATA_IN() && pulse_time < 100) {
        pulse_time++;
        Delay_us(1);
    }
    if (pulse_time >= 100) // 超过 100us 未检测到拉低
        return 1;        // 返回 1 表示不存在
    else                    // 检测到拉低
        pulse_time = 0;    // 重置计数器
    while (!DS18B20_DATA_IN() && pulse_time < 240) // 等待 DS18B20 释放
        数据线
    {
        pulse_time++;
        Delay_us(1);
    }
    if (pulse_time >= 240) // 超过 240us 未检测到释放
        return 1;        // 返回 1 表示不存在

```

```

        else

            return 0;          // 返回 0 表示存在
    }

/**
 * @brief 从 DS18B20 读取一位数据
 * @return 读取的数据位
 */
static u8 DS18B20_Read_Bit(void)
{
    u8 dat;

    /* 拉低数据线???????60us */
    DS18B20_Mode_Out_PP();

    /* 拉低数据线???????1us?15us?? */
    DS18B20_DATA_OUT(LOW);

    Delay_us(10);

    /* 设置为上拉输入模式 */
    DS18B20_Mode_IPU();

    // 读取数据线状态
    if (DS18B20_DATA_IN() == SET)
        dat = 1;
    else
        dat = 0;

    /* 等待 45us */
    Delay_us(45);

    return dat;
}

```

```

/**
 * @brief 从 DS18B20 读取一个字节数据
 * @return 读取的字节数据
 */
u8 DS18B20_Read_Byte(void)
{
    u8 i, j, dat = 0;
    for (i = 0; i < 8; i++) {
        j = DS18B20_Read_Bit(); // 读取一位数据
        dat = (dat) | (j << i);
    }
    return dat;
}

/**
 * @brief 向 DS18B20 写入一个字节数据
 * @param dat 写入的字节数据
 */
void DS18B20_Write_Byte(u8 dat)
{
    u8 i, testb;
    DS18B20_Mode_Out_PP();
    for (i = 0; i < 8; i++) {
        testb = dat & 0x01;
        dat = dat >> 1;
        /* 拉低数据线???????60us */
        if (testb) {

```

```

        DS18B20_DATA_OUT(LOW);

        Delay_us(8); // 1us < ?1?? < 15us

        DS18B20_DATA_OUT(HIGH);

        Delay_us(58); // 58us+8us>60us
    } else {

        DS18B20_DATA_OUT(LOW);

        /* 60us < ?0?? < 120us */

        Delay_us(70);

        DS18B20_DATA_OUT(HIGH);

        /* 1us < ???? < ?????? */

        Delay_us(2);

    }

}

}

/**
 * @brief 启动 DS18B20 进行温度转换
 */
void DS18B20_Start(void)
{
    DS18B20_Rst();          // 发送复位脉冲

    DS18B20_Presence();      // 检测存在性

    DS18B20_Write_Byte(0xCC); // 跳过 ROM 匹配

    DS18B20_Write_Byte(0x44); // 发送温度转换命令
}

/**
 * @brief 初始化 DS18B20

```

```

* @return 0 表示初始化成功，1 表示初始化失败
*/
u8 DS18B20_Init(void)
{
    DS18B20_GPIO_Config();

    DS18B20_Rst();

    return DS18B20_Presence();
}

/**
* @brief 获取 DS18B20 的温度值
* @return 测得的温度值（摄氏度）
*/
float DS18B20_Get_Temp(void)
{
    u8 tpmsb, tpmsb;

    short s_tem;

    float f_tem;

    DS18B20_Rst();

    DS18B20_Presence();

    DS18B20_Write_Byte(0xCC); /* 跳过 ROM 匹配 */

    DS18B20_Write_Byte(0x44); /* 发送温度转换命令 */

    DS18B20_Rst();

    DS18B20_Presence();

    DS18B20_Write_Byte(0xCC); /* 跳过 ROM 匹配 */

    DS18B20_Write_Byte(0xBE); /* 读取温度寄存器 */

```

```

    tpls = DS18B20_Read_Byte();
    tpms = DS18B20_Read_Byte();
    s_tem = tpms << 8;
    s_tem = s_tem | tpls;
    if (s_tem < 0) /* 温度为负 */
        f_tem = (~s_tem + 1) * 0.0625;
    else
        f_tem = (s_tem * 0.0625);

    return f_tem;
}

/**
 * @brief 显示当前温度
 */
void show_temp(void)
{
    static float last_temperature = -1000.0f; // 上一次显示的温度
    // 获取当前温度
    temperature = DS18B20_Get_Temp();
    // 判断温度变化是否超过阈值
    if (fabs(temperature - last_temperature) >= 0.1f) { // 变化大于 0.1°
C
        char p[16] = " ";
        // 格式化温度字符串
        sprintf((char *)p, "%.1f", temperature);
    }
}

```

```

    OLED_ShowCC_F16x16(1, 1, 1);

    OLED_ShowCC_F16x16(1, 4, 2);

    OLED_ShowString(1, 6, p);

    OLED_ShowCC_F16x16(1, 11, 0);

    // 通过串口发送温度信息

    Serial_SendString("当前温度");

    Serial_SendString(p);

    Serial_SendString("° C");

    Serial_SendString("\r\n");

    // 更新上次显示的温度

    last_temperature = temperature;

}

// 延迟 1 秒

Delay_ms(1000);

}

/**
 * @file key.c
 * @brief 按键控制模块的实现文件
 *
 * 该文件包含了按键初始化、按键扫描、模式切换以及温度上下限设置的功能实现。
 */

#include "key.h"

// 初始化温度上下限

float upper_limit = 60.2, lower_limit = 20.5;

// 设置模式标志, 0 为正常模式, 1 为设置模式

```

```

uint8_t mode          = 0;
// 设置目标：0 为上限，1 为下限

uint8_t setting       = 0;
// 用于控制闪烁效果的计数器

uint8_t blink_counter = 0;
// 用于保存上次按键的状态

uint8_t lastKeyNum    = 0;

/**
 * @brief 初始化按键
 */
void Key_Init(void)
{
    // 使能 GPIOA 时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;

    // 配置按键引脚（PA1, PA2, PA3, PA4）为上拉输入模式
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;

    GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_1 | GPIO_Pin_2 | GPIO_Pin_3
| GPIO_Pin_4;

    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(GPIOA, &GPIO_InitStructure);
}

/**
 * @brief 获取按键编号
 *
 * @return uint8_t 按键编号，1-4 对应不同的按键

```

```

*/
uint8_t Key_GetNum(void)
{
    uint8_t KeyNum = 0;

    // 检测按键 K1 是否按下
    if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_1) == 0) {
        Delay_ms(20); // 去抖动延迟
        while (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_1) == 0); // 等待
        按键释放
        Delay_ms(20); // 去抖动延迟
        KeyNum = 1;
    }

    // 检测按键 K2 是否按下
    if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_2) == 0) {
        Delay_ms(20); // 去抖动延迟
        while (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_2) == 0); // 等待
        按键释放
        Delay_ms(20); // 去抖动延迟
        KeyNum = 2;
    }

    // 检测按键 K3 是否按下
    if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_3) == 0) {
        Delay_ms(20); // 去抖动延迟
        while (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_3) == 0); // 等待
        按键释放
        Delay_ms(20); // 去抖动延迟
    }
}

```

```

        KeyNum = 3;
    }

    // 检测按键 K4 是否按下
    if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_4) == 0) {
        Delay_ms(20); // 去抖动延迟

        while (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_4) == 0); // 等待
        按键释放

        Delay_ms(20); // 去抖动延迟

        KeyNum = 4;
    }

    return KeyNum;
}

/**
 * @brief 扫描按键状态并处理按键事件
 */
void key_scan(void)
{
    uint8_t KeyNum = Key_GetNum();

    if (KeyNum != lastKeyNum) { // 判断是否发生按键变化
        lastKeyNum = KeyNum;    // 更新上次按键状态

        // K1 键按下切换模式
        if (KeyNum == 1) {
            mode = !mode; // 切换模式
        }

        // 在设置模式下处理其他按键

```

```

if (mode == 1) {
    // K2 键按下切换设置目标
    if (KeyNum == 2) {
        setting = !setting; // 切换设置目标
    }
    // 根据当前设置目标调整上下限
    if (setting == 0) { // 设置上限
        if (KeyNum == 3) {
            upper_limit += 0.1; // 增加上限
        }
        if (KeyNum == 4) {
            upper_limit -= 0.1; // 减少上限
        }
    } else { // 设置下限
        if (KeyNum == 3) {
            lower_limit += 0.1; // 增加下限
        }
        if (KeyNum == 4) {
            lower_limit -= 0.1; // 减少下限
        }
    }
}

}

/**
 * @brief 显示温度上下限

```

```

*/
void show_warn(void)
{
    char upper[16] = "";
    char lower[16] = "";
    // 格式化上限和下限字符串
    sprintf(upper, "%.2lf", upper_limit);
    sprintf(lower, "%.2lf", lower_limit);
    // 根据设置和计数器决定是否显示上限或下限
    if (mode == 1) { // 模式设置为 1，表示需要闪烁
        if (setting == 0) { // 上限闪烁
            if (blink_counter % 2 == 0) {
                OLED_ShowString(2, 6, upper); // 显示上限
                OLED_ShowCC_F16x16(2, 11, 0);
            } else {
                OLED_ShowString(2, 7, "      "); // 上限位置空白
            }
            // 始终显示下限
            OLED_ShowString(3, 6, lower);
        } else { // 下限闪烁
            if (blink_counter % 2 == 0) {
                OLED_ShowString(3, 6, lower); // 显示下限
                OLED_ShowCC_F16x16(3, 11, 0);
            } else {
                OLED_ShowString(3, 7, "      "); // 下限位置空白
            }
        }
    }
}

```

```

        // 始终显示上限
        OLED_ShowString(2, 6, upper);
    }
} else {
    // 如果不需要闪烁，则正常显示上限和下限
    OLED_ShowString(2, 6, upper); // 显示上限
    OLED_ShowCC_F16x16(2, 11, 0);
    OLED_ShowString(3, 6, lower); // 显示下限
    OLED_ShowCC_F16x16(3, 11, 0);
}

blink_counter++; // 每次刷新时增加计数
}

/**
 * @brief 解析并设置温度上下限
 */
void Serial_ParseAndSetLimits(void)
{
    // 添加字符串结束符
    Serial_RxBuffer[Serial_RxIndex] = '\0';
    Serial_RxIndex = 0; // 重置接收缓冲区索引
    // 解析接收到的数据格式
    if (strcmp(Serial_RxBuffer, "T:") == 0) {
        // 使用 sscanf 解析温度上限
        float new_limit;

        int items = sscanf(&Serial_RxBuffer[2], "%f", &new_limit); // 解
析浮点数
    }
}

```

```

        // 如果解析成功
        if (items == 1) {
            upper_limit = new_limit;
        } else {
            Serial_Printf("Error: Invalid Upper Limit Format!\n");
        }
    } else if (strncmp(Serial_RxBuffer, "L:", 2) == 0) {
        // 使用 sscanf 解析温度下限
        float new_limit;
        int items = sscanf(&Serial_RxBuffer[2], "%f", &new_limit); // 解
析浮动数
        // 如果解析成功
        if (items == 1) {
            lower_limit = new_limit;
        } else {
            Serial_Printf("Error: Invalid Lower Limit Format!\n");
        }
    }
}

/**
 * @brief 串口中断服务程序
 */
void USART1_IRQHandler(void)
{
    if (USART_GetITStatus(USART1, USART_IT_RXNE) == SET) {
        // 读取接收到的数据

```

```

        Serial_RxData = USART_ReceiveData(USART1);

        if (Serial_RxData == '\n' || Serial_RxData == '\r') { // 换行或
回 车 表 示 数 据 接 收 完 毕

            // 添加字符串结束符

            Serial_RxBuffer[Serial_RxIndex] = '\0';

            // 设置接收标志位

            Serial_RxFlag                = 1;

            // 解析并设置温度上下限

            Serial_ParseAndSetLimits();

        } else {

            // 存储接收到的数据

            if (Serial_RxIndex < sizeof(Serial_RxBuffer) - 1) {

                Serial_RxBuffer[Serial_RxIndex++] = Serial_RxData;

            }

        }

        // 清除接收中断标志位

        USART_ClearITPendingBit(USART1, USART_IT_RXNE);

    }

}

#include "stm32f10x.h"

#include "stdio.h"

#include "OLED.h"

#include "key.h"

#include "ds18b20.h"

#include "beep.h"

#include "port.h"

```

```

int main(void)
{
    OLED_Init();
    Key_Init();
    DS18B20_Init();
    Buzzer_Init();
    Serial_Init();

    OLED_ShowCC_F16x16(2, 1, 3);
    OLED_ShowCC_F16x16(2, 4, 5);
    OLED_ShowCC_F16x16(3, 1, 4);
    OLED_ShowCC_F16x16(3, 4, 5);
    OLED_ShowCC_F16x16(4, 1, 6);
    OLED_ShowCC_F16x16(4, 3, 7);
    OLED_ShowCC_F16x16(4, 5, 8);
    OLED_ShowCC_F16x16(4, 7, 9);
    OLED_ShowChar(4, 9, ':');
    OLED_ShowCC_F16x16(4, 10, 10);
    OLED_ShowCC_F16x16(4, 12, 11);
    while (1) {
        show_temp();
        show_warn();
        key_scan();
        AlarmJudge();
    }
}

```

班级：专升本电信 23

学号：ZB2305010116

姓名：姜勇

成绩评定表（教师用）

考核内容	分值（分）	得分
1. 分析温度检测、控制的基本方法。	10	
2. 总体方案设计：画出系统框图，描述系统工作原理。	15	
3. 各功能电路设计：设计各单元电路（包括参数计算，电路工作原理），画出各单元电路图和系统电路图。	20	
4. 软件设计：设计各模块的算法，画出总体流程图和各最小模块流程图（传统、N-S 均可）	20	
5. 编写代码的准确性和完整性。	15	
6. 利用 Proteus 软件对系统进行仿真，展示程序运行效果。	20	